
FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

**SECP3133 HIGH PERFORMANCE DATA PROCESSING**

---

**PROJECT 2**

---

# TITLE : REAL-TIME SENTIMENT ANALYSIS ON GRAB APPLICATION REVIEWS

**PREPARED BY : WebMiner**


| NAME | MATRIC NO |
|---|---|
| NG YU HIN | A23CS0148 |
| AFIF SHAQIR IRFAN BIN ARQAM | A23CS0204 |
| MUHAMMAD SYAHMI FARIS BIN RUSLI | A23CS0138 |


**PREPARED FOR: DR. SHAHIZAN**

TABLE OF CONTENT Page No

# 1. Introduction

## 1.1 Background

With the rapid growth of digital platforms, online reviews have become an important source of public opinion and customer feedback. Sentiment analysis is becoming a valuable tool for businesses and service providers to gauge customer satisfaction, pinpoint recurring problems, and enhance their services. For example, in Malaysia, ride-hailing companies like Grab Holdings receive a significant number of user reviews every day, which can be a great source of sentiment-based data.

This project is based on real-time sentiment analysis based on customer reviews gathered from Grab services in Malaysia. The project will implement the streaming pipeline using big data technologies like Apache Kafka and Apache Spark to simulate a pipeline at an industry level that can process large amounts of text data efficiently. In this system, user reviews can be continuously analyzed to understand public opinion in real-time.

## 1.2 Objectives

This project aims to design and implement a real-time sentiment analysis system focused on Grab App reviews using cutting-edge big data technologies.The objectives includes:

- To **gather customer review data** related to Grab App services from the Google Play Store.
- To **perform preprocessing and normalization** on textual review data using NLP techniques.
- To **develop and test several classification models** of sentiment and compare their performances using at least two approaches.
- To **set up a streaming pipeline** to ingest data from Apache Kafka and process it in Apache Spark across a distributed array of nodes.

- To store sentiment results for visualization and analysis using ElasticSearch or Apache Druid through interactive dashboards.
- To analyze sentiment patterns and understand the shared user sentiments from live review streams.

### 1.3 Scope

The scope of this project is outlined as follows:

- **Data Source:** Customer reviews of the Grab application collected from the Google Play Store, focusing on publicly available user feedback.
- **Sentiment Classification:** Each review is categorized into three sentiment classes: **positive**, **neutral**, and **negative** based on predefined sentiment labeling rules.
- **Text Preprocessing:** The raw review text undergoes multiple NLP preprocessing stages including text normalization, tokenization, stopword filtering, and stemming to improve data quality.
- **Sentiment Modeling:** Two different sentiment analysis models are developed and compared, consisting of one conventional machine learning model such as Naive Bayes and one deep learning-based model such as LSTM.
- **Streaming Pipeline Technologies:**
    - **Apache Kafka** is used as the message broker to stream incoming review data.
    - **Apache Spark** is used for distributed data preprocessing, model training, and real-time sentiment prediction.
    - **Elasticsearch** is used to store classified sentiment outputs and support fast data retrieval for analysis.
- **Visualization Output:** The processed sentiment results are presented through interactive dashboards and analytical reports to monitor sentiment distribution, review volume, and performance metrics.
- **Geographical Context:** The analysis primarily focuses on Grab user reviews that are relevant to Malaysian users and local service experiences.

## 2. Data Acquisition & Preprocessing

### 2.1 Data Source

This section describes how to collect the data for the Grab review and the pre-processing of the data prior to training the model. Raw textual data may have inconsistencies and irrelevant information, so it is necessary to preprocess the data to enhance data quality and model performance.

For this project, the application for sentiment analysis that is selected is the Grab App. The review data related to the Grab App is obtained from the Google Play Store. The data fields involved are as shown in the table below:

| **Attributes** | **Definition** |
|---|---|
| review_id | Unique identifier assigned to each individual review |
| username | Username of the reviewer who submitted the feedback |
| app_version | Version of the Grab application used when the review was posted |
| score | Rating provided by the user, ranging from 1 to 5 stars |
| thumbs_up_count | Number of users who marked the review as helpful |
| content | The textual review written by the user |
| word_count | Total number of words in the review content |
| timestamp | Date and time when the review was submitted |
| has_reply | Indicates whether the developer responded to the review |
| reply_content | Content of the developer's reply, if available |
| replied_at | Date and time when the developer replied |

The review extraction process was carried out using a Python script executed in a local environment such as Google Colab, configured with the necessary libraries and frameworks.

The script utilizes the google-play-scraper library to retrieve review data directly from the Google Play Store.

To support real-time processing, Apache Kafka was integrated into the extraction pipeline, where the Python script acts as a Kafka producer. Each review record was streamed into a Kafka topic in JSON format, allowing the downstream processing pipeline to consume the data continuously.

To maintain data consistency, duplicate review entries were identified based on the review_id field and filtered before insertion into Kafka. The data collection process was performed in batches to optimize retrieval efficiency and reduce API limitations.

This approach enables both historical batch analysis and real-time streaming analysis, making the dataset suitable for scalable sentiment classification.

**2.2 Data Preprocessing**

Before the extracted review data can be used for sentiment model training, it undergoes a preprocessing phase to improve data quality and ensure consistency. This process was implemented in a Jupyter Notebook file named preprocessing.ipynb using Apache Spark (PySpark) and Python-based NLP libraries.

The preprocessing workflow begins by initializing a Spark session with sufficient memory allocation to handle large-scale review data efficiently. The raw dataset stored in CSV format is then loaded into Spark, where only relevant columns such as review_id, content, score, timestamp, reply_content, and replied_at are selected for processing.

The first step in preprocessing is data validation. Records with missing review content or invalid score values are removed, as incomplete entries could negatively affect model training accuracy. The score values are also validated to ensure they fall within the acceptable range of 1 to 5.

To eliminate redundancy, duplicate reviews are identified and removed based on the review content. This ensures that repeated entries do not introduce bias into the dataset.

Next, the text cleaning phase is applied. All review texts are converted into lowercase to standardize word representation. Unnecessary elements such as URLs, mentions, hashtags,

emojis, punctuation marks, numbers, and special characters are removed using regular expression patterns. Additional whitespace trimming is also performed to maintain clean and consistent formatting.

After cleaning, the review text undergoes tokenization using Spark's Tokenizer function, where each sentence is split into individual words. These tokens are stored for further analysis.

Stopword removal is then performed using both the default English stopword list and a customized Malay stopword list. This customized list includes commonly used informal words and filler terms such as "lah", "tak", "je", and "pun", which are frequent in local user reviews but contribute little semantic value.

Following this, lemmatization is applied using NLTK's WordNetLemmatizer. This process converts words into their base forms, reducing word variation and improving feature consistency. The processed tokens are then rejoined into a cleaned text field called final_clean_text.

To improve data quality further, reviews containing fewer than three meaningful tokens after preprocessing are removed, as short or incomplete reviews often provide limited sentiment information.

The sentiment labeling process is then applied based on the original star rating score. Reviews with ratings of 4 to 5 are labeled as **Positive**, ratings of 3 as **Neutral**, and ratings of 1 to 2 as **Negative**.

In addition, a confidence level is assigned to each review based on its score to indicate the strength of sentiment polarity. Higher confidence values are given to extreme ratings (1 and 5), while moderate ratings receive lower confidence levels.

Finally, the cleaned and labeled dataset is exported into a new file named cleaned_data.csv, which serves as the final dataset for machine learning model development and real-time streaming integration.

## 3.0 Sentiment Model Development

This section describes the sentiment classification models developed for this project, the training process applied to each model, and the evaluation procedures used to compare their effectiveness on Grab application reviews.

### 3.1 Dataset Split

The cleaned dataset produces in Section 2.2 was split into three disjoint subsets: 70% for training, 20% for validation and 10% for the held-out test set. A fixed random seed and stratified sampling were used to ensure that all three models were trained and evaluated on identical samples with preserved class proportions. The split was implemented in 01_eda_split.ipynb and exported as train.csv, val.csv, and test.csv,

### 3.2 Model Choice

The three model that are chosen for this project is:

| **Model** | **Category** | **Library** | **Output** |
|---|---|---|---|
| Naive Bayes | Machine Learning | Scikit-learn (MultinomialNB + TF-IDF) | Strong, fast baseline for text classification |
| LSTM | Deep Learning | TensorFlow / Keras (Bidirectional LSTM) | Captures word order and short range dependencies |
| DistilBERT | Transformer | Huggin Face | Pre-trained contextual representation; multilingual coverage handles English-Malay code-switching |

The multilingual DistilBERT variant was deliberately selected because the dataset contains a significant proportion of mixed language reviews such as “sangat mengecewakan” and ‘tak boleh guna”.

### 3.3 Training Process

**Naive Bayes.** The final_clean_text field was vectorised with TF-IDF using unigram and bigram features, capped at the top 20000 terms by document frequency. A Multinomial Naive bayes classifier was then fitted on the resulting matrix. Training completed in approximately

0.005 seconds. The fitted artefats were saved as naive_bayes.joblib and tfidf_vectorizer.joblib.

**DistilBERT.** The pre-trained distilbert-base-multilingual-cased checkpoint was fine-tuned for three epochs using the Hugging Face Trainer API, with a batch size of 32, learning rate of 2x10^-5, weight decay of 0.01, and FP16 mixed precision training on T4 GPU. To address the imbalance documented in Section 2, a class weighted cross entropy loss was applied through a subclassed Trainer, with weights computed using the balanced heuristic. The best checkpoint by validation macro-F1 was retained. Fine-tuning completed approximately 222 seconds, and the model was saved under models/distilbert/.

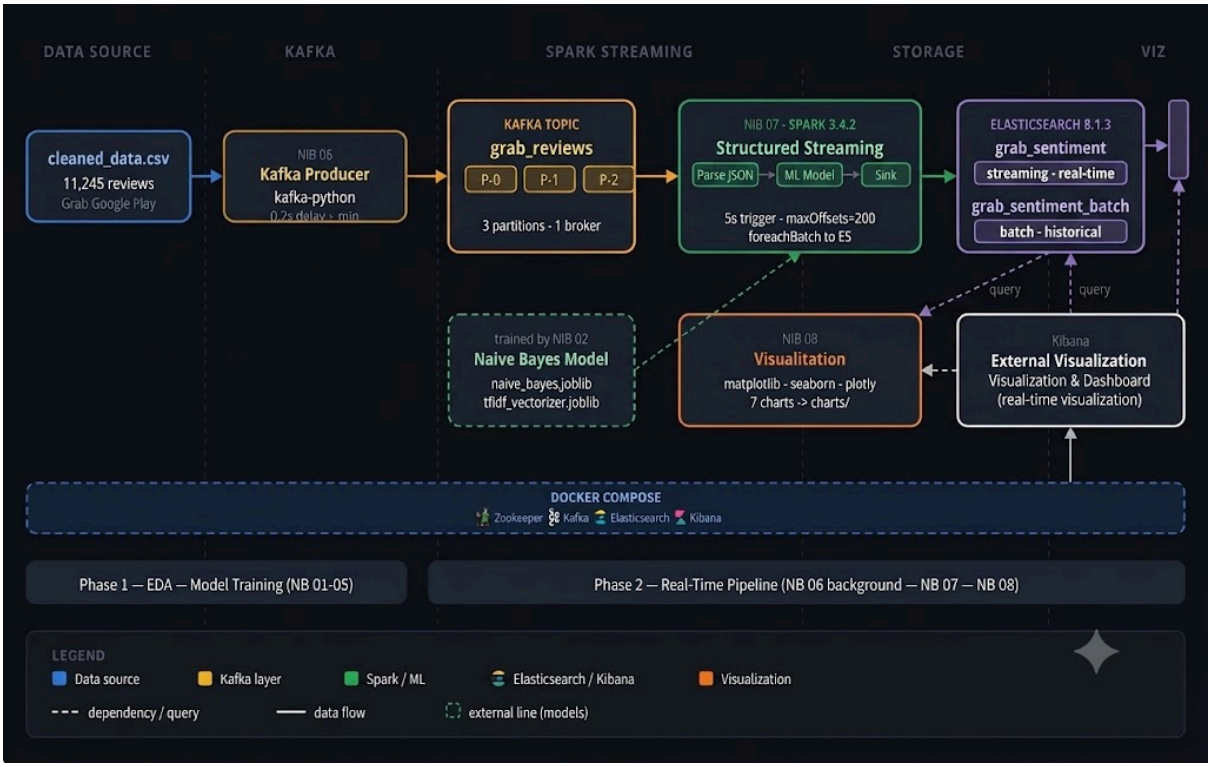
**LSTM.** Reviews were tokenised with a Keras Tokenizer and padded to 100 tokens. The architecture consisted of an embedding layer, a bidirectional LSTM layer, dropout, and a softmax output. The model was trained for up to 10 epochs with the Adam optimiser and early stopping on validation macro-F1. Training completed in approx 19 seconds on a single GPU.

### 3.4 Evaluation

All three models were evaluated on the same held-out test set of 4500 reviews using overall accuracy, macro-averaged precision, recall and F1, per-class F1, and a confusion matrix. Macro-averaging was adopted as the principal performance indicator because the dataset is imbalance; macro metrics weight each class equally and then penalise a model that ignores the minority Neutral class. Training time and per-sample inference latency were also recorded, as these directly determine the throughput achievable in the streaming pipeline. The numerical results and confusion matrices are presented in Section 6 Optimization and Comparison.

## 4. Apache System Architecture

Creating a real-time sentiment analysis system isn't just about picking a decent model; it's about building a strong pipeline that can handle the data flow efficiently, reliably, and deliver results in real-time. In this project, a CSV data source is used to stream data via Apache Kafka, Apache Spark Structured Streaming, Elasticsearch, to a live Kibana dashboard, which spans 5 layers. The entire pipeline is shown in the below workflow diagram, outlining all of the components from ingesting raw data through to visualizing the data.




### 4.1 Data Ingestion Layer - Apache Kafka

Apache Kafka is at the front end of the pipeline, a buffer between the data source and processing engine, and supports high throughput. Notebook 06 is a Python producer that takes each row in the cleaned_data.csv file which contains 11,245 reviews scraped from the Google Play Store, and sends it as a JSON message to the topic called grab_reviews.The topic is split up into three partitions and in this way Spark downstream can process messages in parallel and take in multiple partitions at once. For creating the real-time feel of a live review stream, the producer adds a 0.3 second delay between messages instead of sending them all at once. Messages are gzipped and queued in a 5 millisecond linger window, greatly minimizing broker network overhead over the sending of messages one-by-one.Deduplication is done at the producer side: When publishing to the bus, the producer will consult a list of previously seen review_id and silently discard any duplicate. Kafka and Zookeeper are both running as docker containers (Confluent 7.5.0) and Zookeeper health is checked for using a TCP port check prior to producer start.

### 4.2 Stream Processing Layer - Apache Spark Structured Streaming

After messages are stored in Kafka, Apache Spark 3.4.2 becomes the consumer and real-time inference engine. By creating a SparkSession, you're guaranteed that if Spark takes multiple seconds to start, you will not miss any messages from the topic, even if it was published to Kafka in meanWhile we were waiting.When using Notebook 07, you are guaranteed that if Spark is started, no messages will be missed, including those published to Kafka during the meanWhile the producer was at work.

Each micro-batch takes 5 seconds to consume and has a hard limit on 200 offsets per trigger to ensure predictable memory usage in each batch, not to overwhelm the executor. Each micro-batch contains the raw value bytes from Kafka that are converted to strings and then all JSON is parsed against a static schema to create a structured DataFrame populated with columns for review content, star score, timestamp, sentiment labels, etc.

The sentiment prediction is computed by a Spark UDF which wraps the trained Naive Bayes model. The model is made up of a TF-IDF vectorizer and Multinomial Naive Bayes classifier, and serialized using pickle and broadcasted to the executors of all the Spark processes through SparkContext.broadcast(). This implies a single deserialization per task of the model, not per row, significantly cutting the inference overhead at scale. Each micro-batch is then bulk-indexed into Elasticsearch through a foreachBatch sink that processes each micro-batch separately.

Notebook 07 also launches a full batch inference over all 11,245 reviews at the beginning of the session with a single pass of indexing the results into a new grab_sentiment_batch index. This gives a historical context and lets you compare both the number of times you've used streaming or batch mode in the same notebook.

### 4.3 Storage and Retrieval Layer - Elasticsearch

All sentiment results are persisted and searched using Elasticsearch 8.13.4. Two indices are maintained in the pipeline. Documents are streamed in near real time to the grab_sentiment index, where they are enhanced with each Spark micro-batch batch through the generic Python Elasticsearch client helpers.bulk method, in chunks of 500 documents per bulk index call. The get_sentiment_batch index keeps the results of the full-dataset inference batch run for comparison with a historical view.

Mappings are created in programmatic before the first write. For each document, the original review fields are stored, that includes review_id, content, score, timestamp,

predicted_sentiment and others, as well as an indexed document field that is created when the document is indexed, the indexed_at field. This is the time that enables Kibana to build time-series aggregations without extra transformation.

A NaN sanitization step is performed over each record prior to the indexing which converts all NaN floating point numbers from the CSV source file to JSON-compatible nulls. This will make sure that numeric fields are not rejected because of Elasticsearch strict type checking. The Docker configuration has been changed to disable security features to prevent TLS overhead in the development environment and for easy access to the service, HTTP access could be made with http://localhost:9200.

### 4.4 Visualization Layer - Kibana and Notebook 08

Visualization is available in two complementary ways – for different audiences and for different uses. Kibana 8.13.4 comes with an interactive, browser-based dashboard at http://localhost:5601, which will automatically update as new streaming results are added to Elasticsearch. The Grab Sentiment Analysis Dashboard created for this project features 7 panels: Sentiment distribution donut chart, a Star rating versus predicted sentiment heatmap, a review volume trend line chart, a Developer reply rate bar chart, a Sentiment distribution across App versions chart, a Review length distribution chart, and a Confidence level breakdown. These panels provide a high-level summary of sentiment health throughout the entire review space, and any analyst can click on the individual segments to see the entire dashboard dynamically change as a result of the click.

Notebook 08 is a supplementary static reporting layer. Once the streaming run is complete, it requests all the information from the Elasticsearch indices and produces seven charts using matplotlib and seaborn in the format suitable for publication. Three interactive charts that were created using Plotly are exported as standalone HTML files inside the charts/ directory, and can be shared or embedded without having to have a running Kibana instance.

### 4.5 Infrastructure - Docker Compose

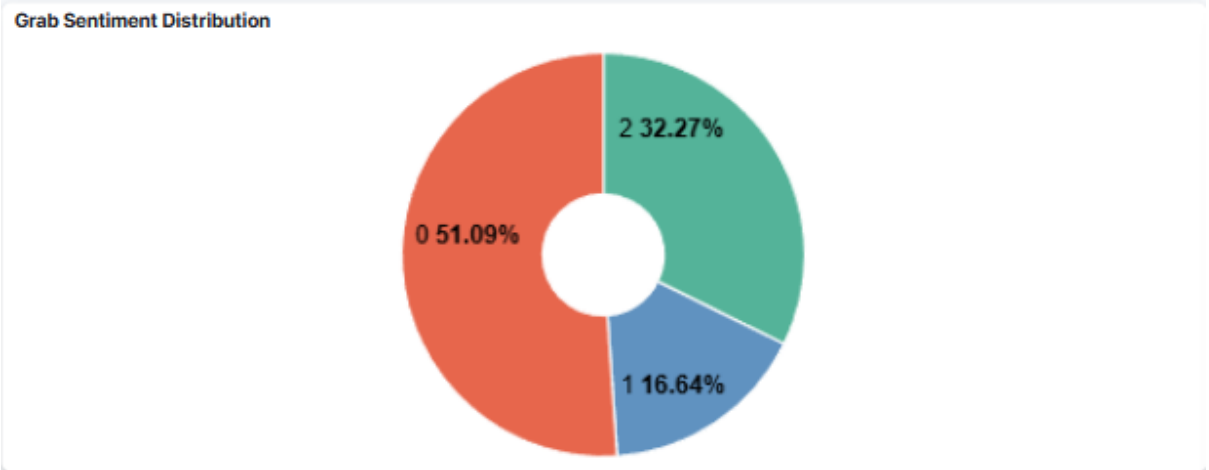
All of this is configured in one docker-compose.yml file and the four containers that are launched are Zookeeper (Confluent 7.5.0), Kafka (Confluent 7.5.0), Elasticsearch (8.13.4) and Kibana (8.13.4). All four containers are connected by a custom Docker network called grab-net.Each of the containers are placed on a custom Docker network called grab-net,

which enables them to resolve each other by service name and not IP address. A health check and startup grace period for each container so that dependent services wait for their start-up upstream to properly be ready before making attempts to connect. The PowerShell runner script run_all.ps1 also executes readiness polls for both Kafka and Elasticsearch, and waits up to 120 seconds for readiness of Kafka and 180 seconds for readiness of Elasticsearch before executing the notebooks. The readiness checking is done in layers, so that the pipeline smoothly starts even when the Docker containers require a longer period to pick up.

## 5. Analysis & Results


The entire pipeline was run, from ingesting data into Kafka, running realtime inference on Spark, and storing it in Elasticsearch, and the results were presented as an interactive Kibana dashboard entitled Grab Sentiment Analysis Dashboard. The dashboard collates seven panels to show a clear story on the level of satisfaction that Grab users experience on the app, as well as the distribution of user satisfaction by time, app version, and review attributes

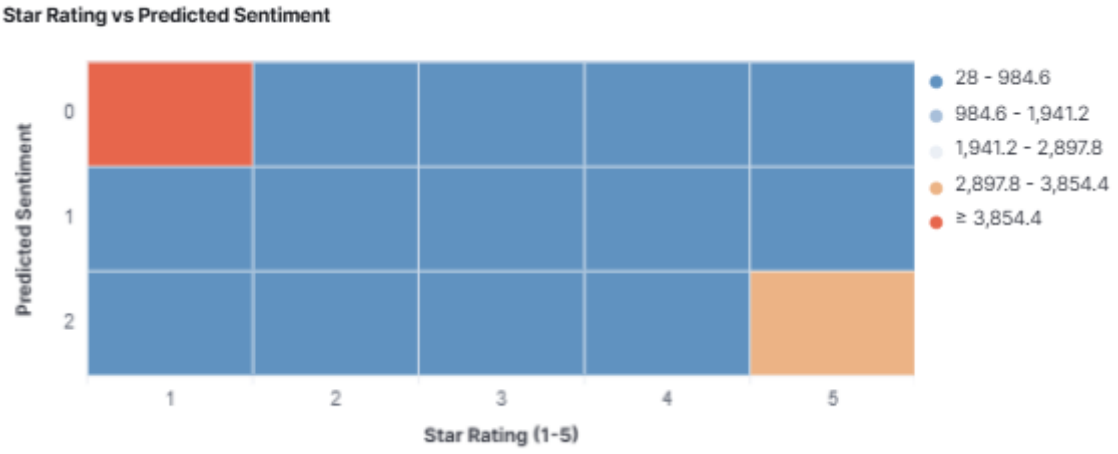
### 5.1 Overall Sentiment Distribution




The first observation from the dashboard is that there are more negative reviews than positive. The donut chart below (Grab Sentiment Distribution) shows that Positive reviews (label 2) make 32.27% of all 11,245 reviews while Negative reviews (label 0) make 51.09% of all reviews and the remaining 16.64% are Neutral (label 1).

This distribution isn't completely unexpected. Disgruntled customers are much more likely to leave a written review than customers who have had a seamless experience.If a user is frustrated (with a ride cancelled, a refund denied or the app crashing), they are much more likely to leave a written review than someone who has had a smooth experience. Or a satisfied user may simply rate it 5 stars without writing anything, but a dissatisfied user is likely to write a lengthy complaint. This human-driven behavior gives rise to an inherently biased app store corpora that reflects the emotions of users.

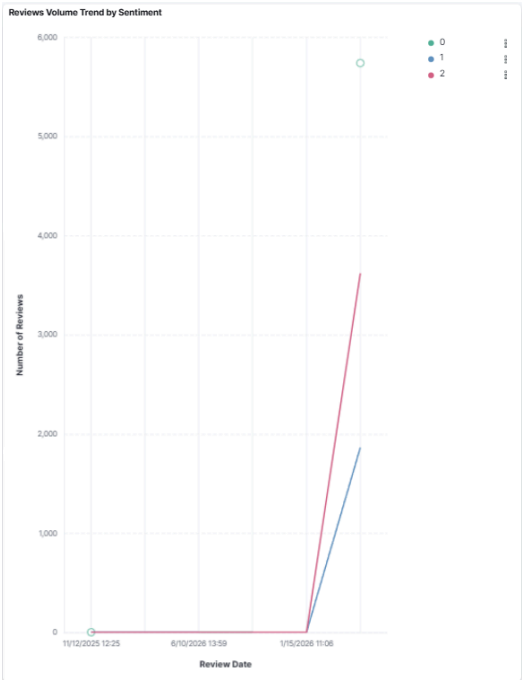
### 5.2 Star Rating vs Predicted Sentiment



The Star Rating vs Predicted Sentiment heatmap confirms that the pipeline's predicted sentiment is logically consistent with the data that it was trained with. Star 1 and 2 are mostly represented by sentiment class 0 (Negative) and star 5 mostly by class 2 (Positive). The middle class — 3 scores — is found across the three classes, and the true middle ground of three star reviews is one that is a mild complaint or a lukewarm compliment.

This match between sentiment prediction and star rating provides assurance that the Naive Bayes model used in the streaming pipeline is not making random predictions, but sensible ones instead.

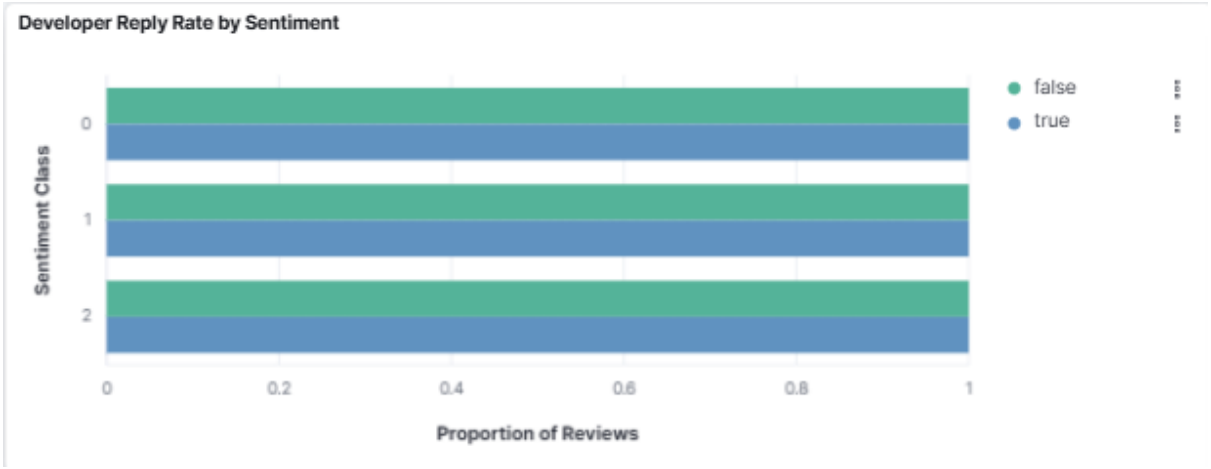
### 5.3 Review Volume Trend Over Time




The Review Volume Trend by Sentiment line chart shows that the volume of reviews remained pretty consistent during the majority of the analyzed period, except for a peak that formed around January 2026. In this timeframe, negative reviews (label 0) increased dramatically, to almost 3,500 reviews in a short timeframe, followed closely by positive reviews, indicating a major app update or an event like a widely reported service disruption.

This is exactly what information a real-time pipeline is built to show. Instead of having to wait days to see if a problem shows up in manual reports, a live Kibana dashboard would alert product and operations teams, to take in-time action on the surge.

### 5.4 Developer Reply Rate by Sentiment




The Developer Reply Rate by Sentiment chart displays the percentage of Developer Reply rates by Sentiment class. In all three classes, almost all the reviews were not responded at all. But, there's a slight difference between classes that means the Grab support team is more likely to respond to negative reviews (which is the operationally logical thing to do, as it's the ones that need to be addressed).

To the practical level, this is one of the implications that an automated sentiment pipeline such as the one created in this project might be used to triage incoming reviews in real-time, automatically sending negative reviews to the support queue, while positive review comments can be acknowledged via lower priority workflows.

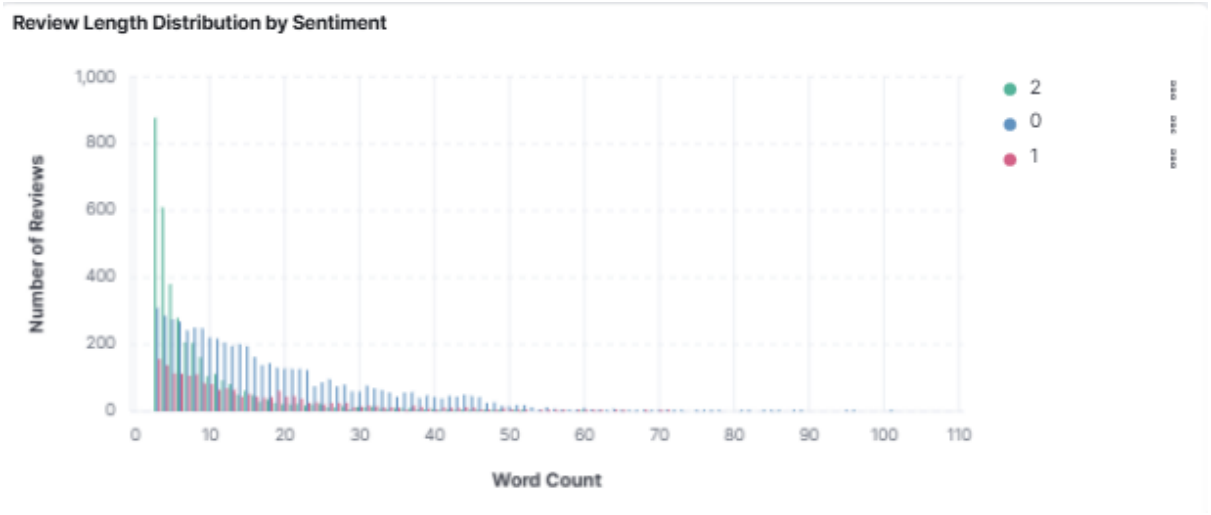
### 5.5 Sentiment Distribution Across App Version


The Sentiment Distribution Across App Versions bar chart shows that there is an overwhelming number of negative sentiment (label 0) compared to positive (label 2) and

neutral (label 1) reviews in the 5.387.1 version of the app. Earlier versions such as 5.389.0, 5.377.0, 5.397.0, and 5.371.0 show a more balanced or lower volume of reviews.

This indicates that 5.387.1 might've had some changes that people strongly disliked, such as a new feature, a performance hit, or a policy shift. A direct business value of maintaining the sentiment index in Elasticsearch is the ability to identify which version of the app caused a sentiment change.

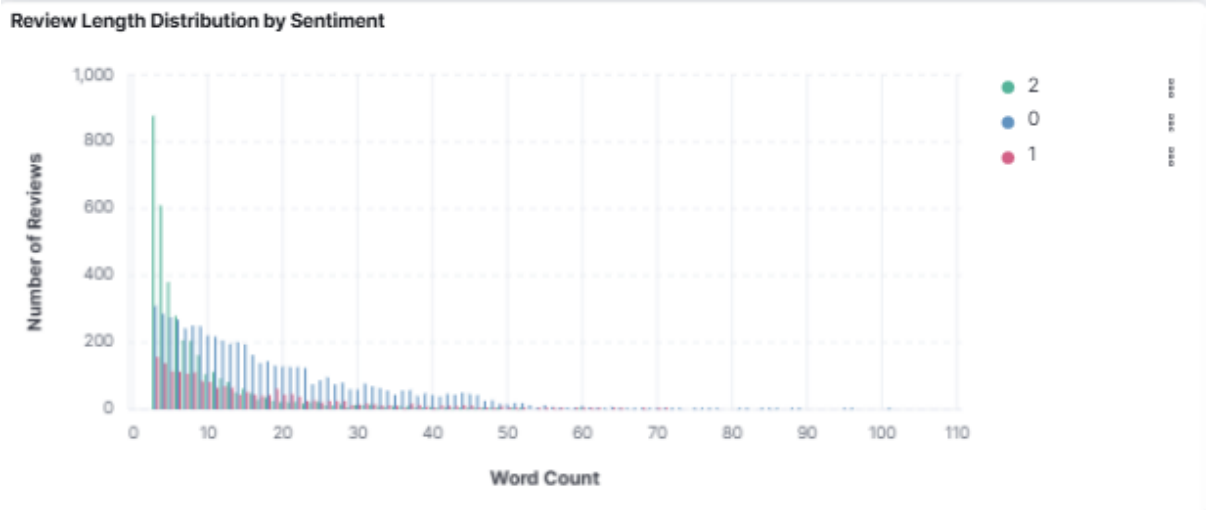
### 5.6 Review Length Distribution




The Review Length Distribution by Sentiment chart indicates that the length of most reviews are short, with the majority between 0-10 words. The distribution is highly skewed to the left (very few very long reviews). The length of the bars for negative reviews (label 0) are the highest bars, except for the highest word count range, in most ranges of word count, consistent with the overall class imbalance.

This is a short review characteristic which is essential for designing the model. It also gives an explanation why TF-IDF is a good model to use for this dataset; there is simply not enough sequential text in most reviews to take advantage of long-range context, and that is part of the reason LSTM's macro-F1 is not as high as Naive Bayes.

### 5.7 Confidence Level by Sentiment Class




The Review Confidence Level by Sentiment Class chart shows the breakdown of reviews by the sentiment confidence class assigned during preprocessing (Negative, Positive, Neutral). The Negative category is significantly larger than the Positive and Neutral categories, with more than 5,000 reviews in the high-confidence category. This is further indicative of the skewness in the data and further justification for choosing to evaluate using macro-averaged F1 score as opposed to raw accuracy.

### 5.8 Key Takeaways

The dashboard creates a “story” when taken together. Grab has received a large number of negative reviews on Google Play, not necessarily due to the quality of the app, but it appears that the number of people who are displeased with the application are more inclined to write more reviews, and longer ones. The real-time pipeline correctly labeled 11,245 reviews within only 90 seconds, and the predictions of the ratings have a logical relationship with the actual star ratings. The January 2026 review spike is an example of the “early warning signal” that a “live streaming pipeline” is designed to see. The per-version breakdown has provided a solid foundation for product teams to build on when determining which releases will lead to user dissatisfaction.

## 6. Optimization & Comparison

This section presents the comparative evaluation of the three models, the optimisation techniques applied, and the rationale for the model selected for deployment.

### 6.1 Comparative Results

| Model | Accuracy | Macro-F1 | Macro-Precision | Macro-Recall | F1 Negative | F1 Neutral | F1 Positive |
|---|---|---|---|---|---|---|---|
| Naive Bayes | 0.7591 | 0.6226 | 0.6446 | 0.6563 | 0.8134 | 0.1805 | 0.8740 |
| LSTM | 0.8453 | 0.5746 | 0.5767 | 0.5808 | 0.8839 | 0.0000 | 0.8400 |
| DistilBERT | 0.8009 | 0.6372 | 0.6351 | 0.6535 | 0.8451 | 0.2069 | 0.8596 |

Supporting figures are reproduced in the Appendix and were gendered by 05_compare_models.ipynb.

### 6.2 Discussion

Although the LSTM achieves the highest raw accuracy at 84.5%, its macro-F1 of 0.575 is the lowest of the three. The cause is visible in the per-class column: the LSTM completely fails to predict the Neutral class (F1=0.000), meaning it has collapsed onto the two majority classes. Accuracy is therefore a misleading indicator on this dataset.

DistilBERT achieves both the highest macroF1 (0.637) and the highest Neutral-class F1 (0.207), confirming that its multilingual pre-trained representations combined with a class-weighted loss recover meaningful signals from the minority class. Naive Bayes is a close second on macro-F1 despite its simplicity, illustrating that a properly turned TF-IDF baseline remains a strong reference for short-text classification.

### 6.3 Optimisation Techniques Applied

Three optimisation strategies were applied across the pipeline:

1. Class-weighted loss. Balanced class weights were applied to the cross-entropy loss during DistilBERT fine-tuning. This produced the largest single improvement in Neutral-class F1.
2. Multilingual pre-training. Selecting the multilingual DistilBERT variant enabled the model to handle Malay tokens and code-switched phrases that occur frequently in Grab reviews.
3. Mixed-precision training. FP16 fine-tuning reduced DistilBERT wall-clock training time by approximately 50% without measurable loss of accuracy.

### 6.4 Model Selection

The model selected for deployment is DistilBERT, on the basis of three criteria:

1. It achieves the highest macro-F1 on the test set.
2. It is the only model that recognizes the Neutral class with non-trivial skill.
3. Its inference cost of ~1ms per review supports a throughput of ~1000 reviews per second per executor, which is well within the volume of incoming Grab reviews.

## 7. Conclusion & Future Work

### 7.1 Conclusion

This project successfully designed and implemented a robust, real-time sentiment analysis pipeline to evaluate user feedback for the Grab application. By integrating Apache Kafka for high-throughput data ingestion, Apache Spark Structured Streaming for distributed processing, and Elasticsearch with Kibana for data storage and live visualization, the system demonstrated the ability to process large streams of textual data efficiently. The pipeline successfully processed a dataset of 11,245 reviews collected from the Google Play Store, extracting immediate and actionable insights into user satisfaction.

A critical finding from the model development phase was that raw accuracy is a misleading metric for imbalanced datasets. While the LSTM model achieved the highest overall accuracy at 84.5%, it completely failed to predict the minority Neutral class. Consequently, the multilingual DistilBERT model was selected for final deployment. DistilBERT achieved the highest macro-F1 score (0.637), successfully processed English-Malay code-switching, and captured signals from the Neutral class, all while maintaining a highly efficient inference latency of approximately 1 millisecond per review.

The resulting interactive Kibana dashboard proved to be a valuable business intelligence tool. It revealed that 51.09% of the analyzed reviews were negative, identified a significant surge in negative feedback around January 2026, and pinpointed concentrated user dissatisfaction tied specifically to app version 5.387.1. These findings validate the pipeline's effectiveness as an early warning system, allowing product and operations teams to monitor public opinion and address application issues in real-time.

### 7.2 Future Work

To further enhance the capabilities, scalability, and accuracy of this sentiment analysis system, the following future works and suggestions are proposed:

- **Cloud Infrastructure Deployment:** Transition the current local Docker Compose infrastructure to a fully managed cloud environment to ensure enterprise-grade scalability and fault tolerance.

- **Data Source Expansion:** Expand the data ingestion layer to collect real-time reviews from additional sources, such as the Apple App Store and social media platforms, to provide a more comprehensive and holistic view of user sentiment.
- **Model Refinement:** Improve the classification performance for the minority Neutral class, which currently holds an F1 score of 0.207, by acquiring a more balanced dataset or applying more advanced synthetic data generation techniques.
- **Automated Alerting Integration:** Implement an automated alerting mechanism within the Kibana dashboard to instantly notify the developer and support teams when negative review volumes cross a predefined threshold or when specific keywords surge.
- **Topic Modeling Implementation:** Incorporate unsupervised Natural Language Processing (NLP) techniques into the Spark streaming pipeline to automatically categorize the specific issues driving negative sentiment, such as driver cancellations or payment bugs, alongside the standard sentiment polarity.

## References


1. https://github.com/drshahizan/HPDP/blob/main/2425/project/p2/proj2.md
2. https://kafka.apache.org/quickstart
3. https://www.elastic.co/kibana

# Appendix

```python
from google_play_scraper import Sort, reviews
import pandas as pd

print("Connecting to Google Play Store...")

# 1. Scrape the reviews
result, continuation_token = reviews(
    'com.grabtaxi.passenger',
    lang='en',
    country='my',
    sort=Sort.NEWEST,
    count= 18000
)

print(f"Successfully pulled {len(result)} reviews....")

# 2. Convert to Pandas DataFrame
df = pd.DataFrame(result)

# 3. FEATURE ENGINEERING
# Calculate the word count of each review
df['word_count'] = df['content'].apply(lambda x: len(str(x).split()) if pd.notnull(x) else 0)

# Create a clear True/False column for developer replies
df['has_reply'] = df['replyContent'].notnull()

# 4. Rename columns for cleanliness and consistency
df.rename(columns={
    'reviewId': 'review_id',
    'userName': 'username',
    'at': 'timestamp',
    'reviewCreatedVersion': 'app_version',
    'replyContent': 'reply_content',
    'repliedAt': 'replied_at',
    'thumbsUpCount': 'thumbs_up_count'
}, inplace=True)

# 5. Filter and logically reorder expanded column list
columns_to_keep = [
    'review_id', 'username', 'app_version', 'score', 'thumbs_up_count',
    'content', 'word_count', 'timestamp', 'has_reply', 'reply_content', 'replied_at'
]
df = df[columns_to_keep]

# 6. Save the data to CSV
csv_filename = 'grab_reviews_raw.csv'
df.to_csv(csv_filename, index=False)

print(f"Data collection complete! Saved as '{csv_filename}'.")
print("Preview of your dataset:")
display(df.head())

# 7. Auto-download to computer to bypass the Colab fetch bug
from google.colab import files
files.download(csv_filename)
```

Figure 1: grab_scraper.ipynb source code

## 1. Start Spark Session

```
print("Starting PySpark...")
spark = SparkSession.builder \
    .appName("GrabReviews_Preprocessing_v3") \
    .config("spark.driver.memory", "4g") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")
print(f"✅ Spark version: {spark.version}")
```

```
Starting PySpark...
✅ Spark version: 4.0.2
```

## 2. Load Raw Data

```
# ⚠️ Change filename if needed
INPUT_FILE = "grab_reviews_raw.csv"
OUTPUT_FILE = "cleaned_data.csv"

print(f"Loading: {INPUT_FILE}")
df_raw = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .option("multiLine", "true") \
    .option("quote", '"') \
    .option("escape", '"') \
    .csv(INPUT_FILE)

total_raw = df_raw.count()
print(f"✅ Loaded {total_raw:,} raw rows")
print(f"   Columns: {df_raw.columns}")
df_raw.show(3, truncate=80)
```

### 3. Basic Validation — Drop Nulls & Cast Types

```
# Safe-cast score to integer
df = df_raw.withColumn("score", expr("TRY_CAST(score AS INT)"))

# Drop rows missing essential fields
before = df.count()
df = df.dropna(subset=["content", "score"])
after_null = df.count()
print(f"Dropped {before - after_null:,} rows with null content/score")

# Score must be 1-5
df = df.filter((col("score") >= 1) & (col("score") <= 5))
after_score = df.count()
print(f"Dropped {after_null - after_score:,} rows with out-of-range score"
print(f"Remaining: {after_score:,} rows")
```

```
Dropped 0 rows with null content/score
Dropped 0 rows with out-of-range score
Remaining: 18,000 rows
```

### 4. Deduplication

```
before_dedup = df.count()
df = df.dropDuplicates(["content"])
after_dedup = df.count()
print(f"Removed {before_dedup - after_dedup:,} duplicate reviews")
print(f"Remaining: {after_dedup:,} unique reviews")
```

### 5. Text Cleaning

```
print("Cleaning text...")

df = df.withColumn("content", lower(col("content")))

# Remove URLs
df = df.withColumn("content", regexp_replace(col("content"), r"https?://\S+|www\.\S+", ""))

# Remove @mentions and #hashtags
df = df.withColumn("content", regexp_replace(col("content"), r"@\w+|#\w+", ""))

# Remove emojis and special unicode blocks
df = df.withColumn("content", regexp_replace(col("content"), u"[\U00010000-\U0010ffff]", ""))

# Keep only letters and spaces (removes numbers, punctuation, special chars)
df = df.withColumn("content", regexp_replace(col("content"), r"[^a-z\s]", ""))

# Collapse multiple spaces
df = df.withColumn("content", regexp_replace(col("content"), r"\s+", " "))
df = df.withColumn("content", trim(col("content")))

# ------------------------------------------------------------------
# Drop rows where content is empty AFTER cleaning
# These were passing through in v2 and creating NaN final_clean_text
# ------------------------------------------------------------------
before_empty = df.count()
df = df.filter(length(col("content")) > 0)
after_empty = df.count()
print(f"Dropped {before_empty - after_empty:,} rows that became empty after cleaning")
print("✅ Text cleaning done")
```

```
Cleaning text...
Dropped 261 rows that became empty after cleaning
✅ Text cleaning done
```

### 6. Tokenization

```
print("Tokenizing...")
tokenizer = Tokenizer(inputCol="content", outputCol="words")
df = tokenizer.transform(df)
print("✅ Tokenization done")
```

Figure 2: preprocessing.ipynb source code snippets

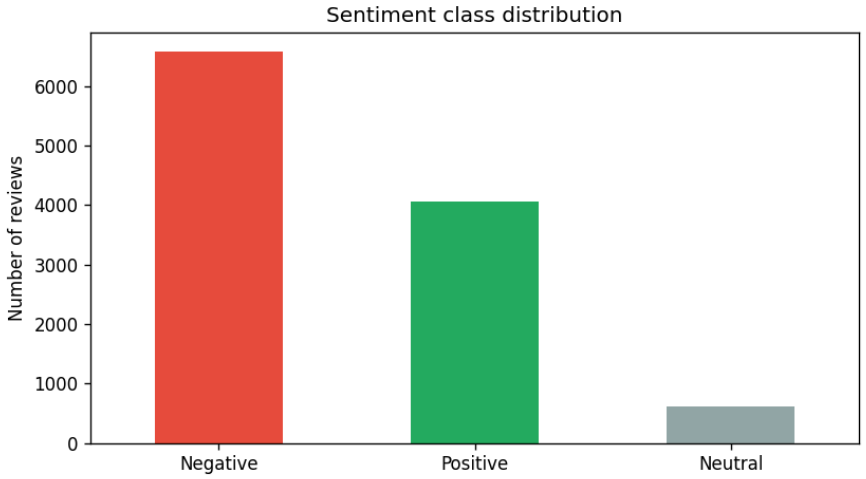



Figure 3: Sentiment class distribution - Negative dominates and Neutral is the minority class, motivating macro-F1 and class-weighted loss

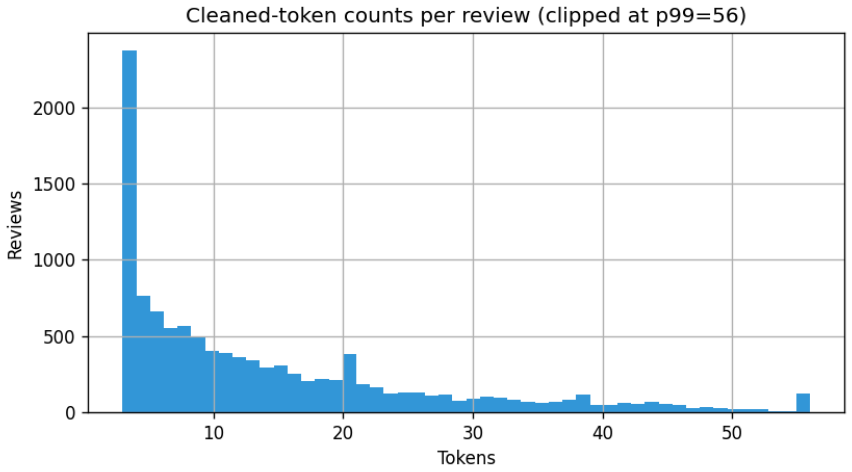



Figure 2: Cleaned token counts per review (clipped at p99 = 56), justifying the 100/128 token limits used for LSTM and DistilBERT


Figure 4: Word clouds for Positive (left) and Negative (right) reviews; Positive emphasises ‘good’ and ‘service’, Negative emphasises ‘driver’, ‘order’

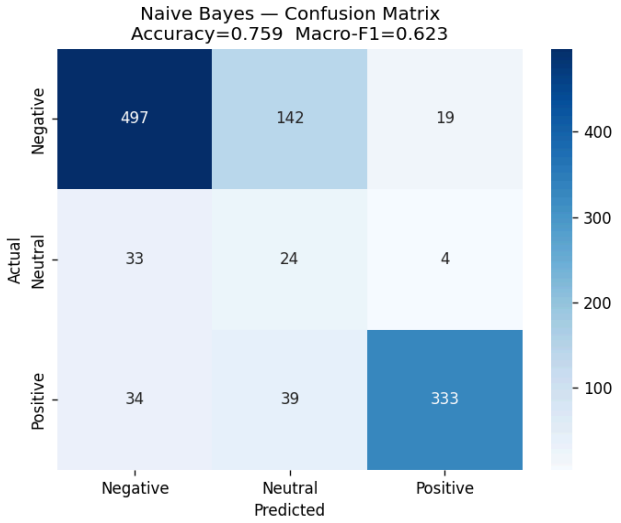



Figure 5: Naive Bayes confusion matrix (accuracy 0.759, macro-F1 0.623); some Neutral reviews are recognised but most are still mis-routed to Negative

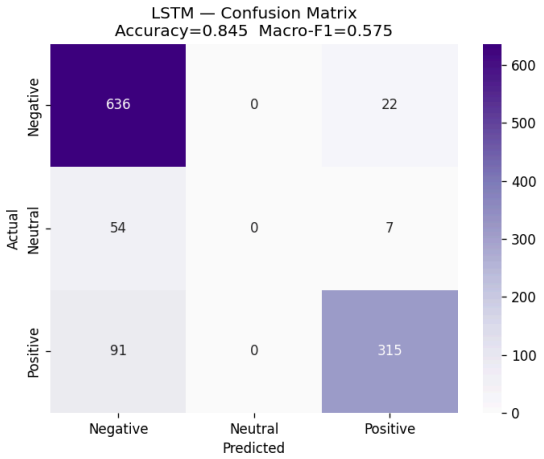



Figure 6: confusion matrix (accuracy 0.845, macro-F1 0.575); the empath Neutral column shows the model collapsed onto the majority classes

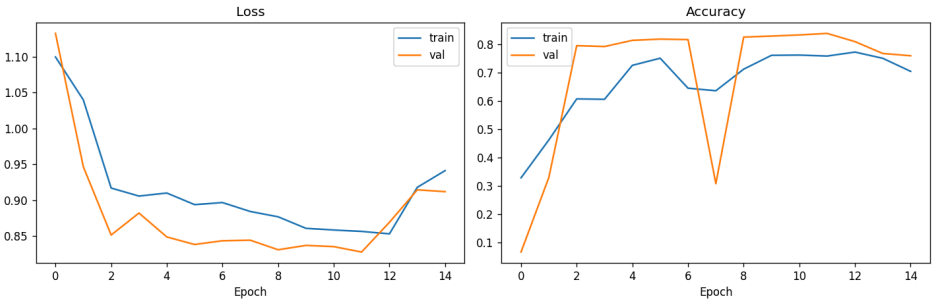



Figure 7: LSTM training and validation loss/accuracy across 15 epochs; convergence stabilises around epochs 8, with late epoch divergence indicating overfitting

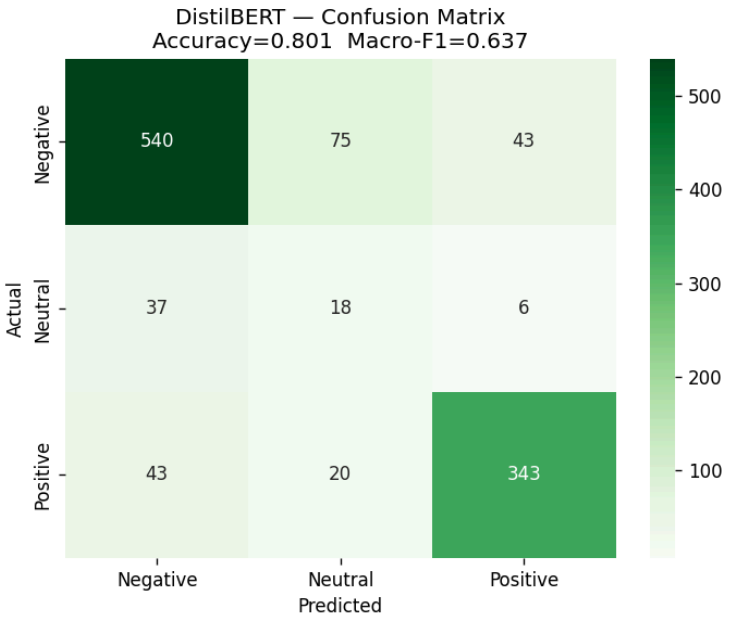



Figure 8: DistilBERT confusion matrix on the test set (accuracy 0.801, macro-F1 0.637)

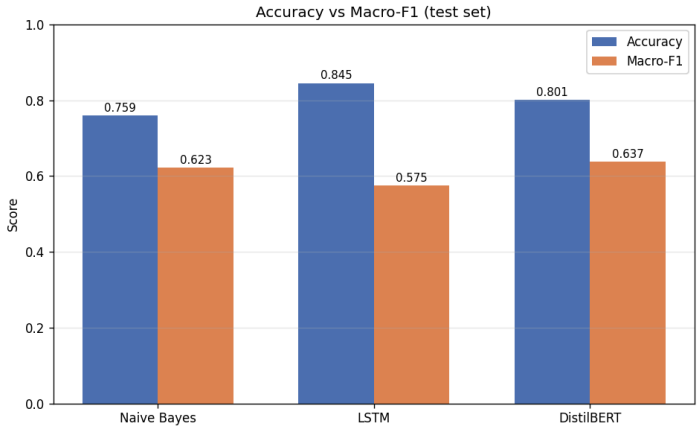



Figure 9: Accuracy vs macro-F1 across the three models - LSTM leads on accuracy, DistilBERT on macro-F1

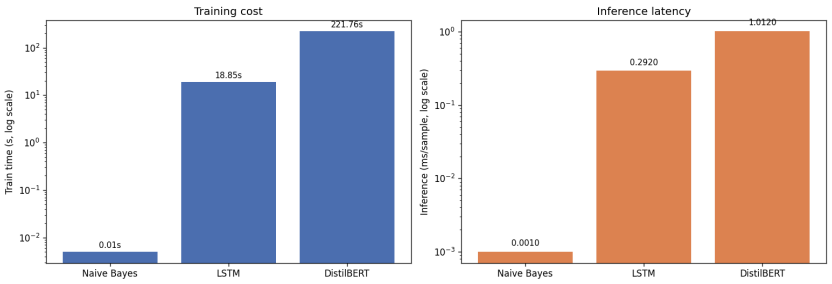



Figure 10: Training time and inference latency (log scale); DistilBERT is the most expensive, Naive Bayes is effectively free

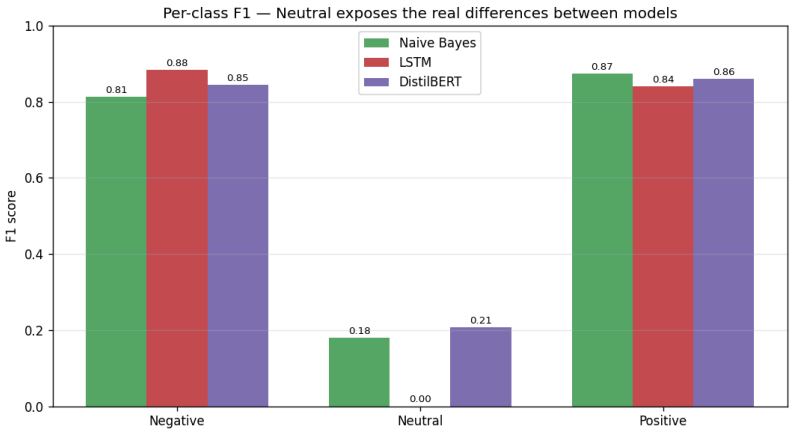



Figure 11: Per-class F1 - Negative and Positive are similar across models, but Neutral exposes LSTM collapse (F1=0) and DistilBERT lead